

PRD: Palate

1. Product Overview

Product Name

Palate

One-Line Description

Palate helps users discover, rank, save, compare, and support local restaurants, cafes, dessert shops, and food businesses based on their personal taste, trusted reviews, active deals, and dining history.

Core Concept

Palate is a local food discovery app centered around personal rankings and taste-based recommendations. Users do not just rate restaurants with generic stars. They compare places head-to-head, build ranked lists, discover nearby food businesses, claim local deals, and receive recommendations based on what they actually like.

Example:

"I just tried a new ramen spot. Was it better than my current #3 ramen place?"

The app uses pairwise comparison to place the restaurant into the user's personal ranking, then uses that ranking data to recommend better local options.

Competition Goal

Build a standalone FBLA Coding & Programming application that demonstrates:

- local business discovery
- food business categories
- ratings and reviews
- sorting by reviews and ratings
- bookmarking
- special deals and coupons
- bot-prevention verification
- intelligent recommendations
- data analysis and reports
- strong data structures
- clean, modular code
- polished UX

2. Product Positioning

Positioning Statement

Palate is a personalized local food discovery platform that helps users find the best restaurants, cafes, and food businesses for their taste through ranked lists, pairwise comparisons, trusted reviews, and local deals.

What Palate Is

Palate is:

- a local restaurant discovery app
- a personal food ranking app
- a trusted review app
- a local deals app
- a taste profile app
- a food business support platform

What Palate Is Not

Palate should avoid becoming a basic Google Maps clone or a generic restaurant directory. The product must center around personal taste, ranking, comparison, and local support.

Key Differentiator

The key differentiator is **TasteRank**.

Instead of relying mainly on broad public ratings, Palate builds a user's personal food rankings through quick head-to-head comparisons.

Example:

"Was this burger better than Burger Factory?" "Was this cafe better for studying than Vereda?" "Was this sushi spot better value than August 8?"

This creates more personalized recommendations than a basic star rating.

3. Problem Statement

User Problem

People often struggle to decide where to eat because:

- Google reviews are too broad
- restaurant ratings are often similar
- people trust friends more than strangers
- users forget places they liked
- generic ratings do not capture personal taste
- local deals are scattered
- choosing with friends is annoying

Business Problem

Small local food businesses need:

- more visibility
- more repeat customers
- better ways to promote deals
- verified reviews
- insight into what customers like
- ways to compete with larger chains

Product Opportunity

Palate helps users make better food decisions while helping small local food businesses gain visibility through rankings, recommendations, reviews, saves, deals, and analytics.

4. Target Users

Primary User: Food Explorer

A student or young adult who likes trying food spots and wants to remember, rank, and discover better places.

Needs:

- rank restaurants
- find hidden local spots
- save places to try
- claim food deals

- get recommendations based on taste
- compare restaurants easily

Secondary User: Casual Local Customer

A person who wants to quickly find a good place for a specific situation.

Needs:

- lunch under a budget
- cafe for studying
- dessert spot
- group dinner
- date spot
- student discounts
- trusted reviews

Tertiary User: Local Food Business Owner

A restaurant, cafe, bakery, dessert shop, or food truck owner.

Needs:

- post deals
- attract local users
- view saves and claims
- track reviews
- see customer preference trends
- understand which offers work

Competition User: FBLA Judge

Needs:

- quickly understand the app
 - see full prompt alignment
 - see technical depth
 - see intelligent features
 - see reports and data storage
 - see clean UX and working demo
-

5. Core Product Pillars

1. Personal Taste Over Generic Ratings

Palate should focus on what the individual user likes, not only what the crowd rates highly.

2. Pairwise Comparison

Users rank businesses by comparing one place against another. This is easier and more accurate than asking users to assign perfect star ratings.

3. Local Food Business Support

The app should clearly promote local restaurants, cafes, dessert shops, bakeries, and food businesses.

4. Deals and Coupons

Deals should be visible, claimable, and trackable.

5. Verified Reviews

Users should only leave verified reviews after claiming or marking a visit.

6. Reports and Analytics

Users and businesses should both receive useful reports.

6. MVP Scope

Must-Have Features

1. User profile selector
2. Home dashboard
3. Explore local food businesses
4. Category filtering
5. Sort by rating and review count
6. Business profile pages
7. Save/bookmark businesses
8. Save/bookmark deals
9. Reviews and ratings
10. Verified review flow
11. Active deals and coupons

12. Claim deal flow
13. Claim code generation
14. Bot-prevention verification
15. TasteRank pairwise comparison
16. Personal ranked lists
17. Taste profile
18. Recommendation engine
19. Group decision mode
20. User report dashboard
21. Business owner dashboard
22. Business deal creation
23. Business analytics
24. Local data persistence
25. Reset demo data button

Nice-to-Have Features

1. Friend taste similarity
2. Group swipe voting
3. "Where should we eat?" mode
4. Cuisine passport
5. Food mood selector
6. Dark mode
7. Mock map view
8. Export report as CSV
9. Accessibility preference filters
10. Photo placeholders

Out of Scope

Do not build:

- real payments
 - real delivery ordering
 - real reservations
 - real Google Maps dependency
 - real GPS dependency
 - real SMS verification
 - real restaurant onboarding
 - real social media messaging
 - live API dependency
 - full AI chatbot
-

7. Main Product Features

7.1 Home Dashboard

Purpose

Give users quick access to discovery, rankings, deals, and saved places.

Sections

- Greeting
- Main CTA: "Find Food"
- Secondary CTA: "Rank a Place"
- Active deals near you
- Saved places
- Top ranked categories
- Recent visits
- Taste profile summary

Example Home Copy

Header:

What are you craving?

Subtext:

Discover local food spots based on your taste, rankings, friends, and active deals.

Buttons:

- Find Food
 - Rank a Place
 - View Deals
-

7.2 Explore Page

Purpose

Let users browse local restaurants and food businesses.

Business Categories

- Restaurants

- Cafes
- Bakeries
- Dessert shops
- Fast casual
- Food trucks
- Bubble tea
- Pizza
- Burgers
- Sushi
- Ramen
- Breakfast
- Vegetarian
- Student-friendly
- Study spots

Filters

- Category
- Cuisine
- Price range
- Rating
- Distance
- Open now
- Active deals
- Student discounts
- Saved by friends
- Verified reviews
- Good for groups
- Good for studying

Sort Options

- Highest rated
- Most reviews
- Closest
- Best match
- Most saved
- Active deals
- Newest
- Best value

Business Card Fields

Each card should show:

- Business name
- Category/cuisine
- Rating

- Review count
 - Distance
 - Price level
 - Active deal count
 - Saved status
 - Verified badge
 - Top tags
 - View button
 - Save button
-

7.3 Business Profile Page

Purpose

Show complete details for a restaurant or food business.

Sections

1. Header
2. Rating summary
3. Active deals
4. Personal ranking placement
5. Reviews
6. Business details
7. Similar businesses

Header Fields

- Business name
- Cuisine/category
- Rating
- Review count
- Distance
- Price range
- Save button
- Verified badge

Active Deals

Each deal card shows:

- Deal title
- Deal price
- Original price if applicable
- Savings
- Valid time

- Claim button
- Save button
- Remaining claims

Personal Ranking Placement

If the user has ranked this place:

Ranked #2 in your sushi list

If not ranked:

Add this to your personal ranking

Reviews

Show:

- Verified reviews first
- Rating
- Text
- Tags
- Date
- User name
- Verified badge

Sort reviews by:

- Newest
- Highest rating
- Lowest rating
- Most helpful

Filter reviews by:

- Student-friendly
 - Good value
 - Good for groups
 - Study-friendly
 - Fast service
 - Vegetarian options
-

7.4 TasteRank Pairwise Comparison

Purpose

Create personal restaurant rankings through quick comparisons.

User Flow

1. User selects a business they visited.
2. App asks comparison questions.
3. User answers better, worse, about same, or skip.
4. App inserts the business into the correct ranked list.
5. User sees updated ranking.

Example Prompt

Was The Burger Room better than Burger Factory?

Buttons:

- Better
- Worse
- About the same
- Skip

Ranking Categories

Users can rank by:

- Overall
- Burgers
- Sushi
- Ramen
- Cafes
- Dessert
- Bubble tea
- Study spots
- Cheap eats
- Date spots
- Group dinner
- Best value

Algorithm

Use binary insertion.

If a user has n businesses in a ranked list, a linear comparison may require up to n comparisons. Binary insertion reduces this to about $\log$ base 2 of n comparisons.

Example: A list with 64 restaurants needs about 6 comparisons instead of up to 64.

Acceptance Criteria

- User can select category to rank in.
 - User can compare businesses.
 - App updates ordered ranking.
 - Ranking persists after refresh.
 - App explains ranking update.
 - User can view ranking later.
-

7.5 Recommendation Engine: PalateMatch

Purpose

Recommend businesses based on user taste, rankings, preferences, ratings, deals, and context.

Input Signals

- Cuisine preference
- Past ratings
- Pairwise rankings
- Saved businesses
- Claimed deals
- Friend activity from seeded data
- Price preference
- Distance preference
- Category selected
- Active deals
- Business rating
- Verified review count

Recommendation Score Formula

Score =

0.25 taste score

- 0.20 category/cuisine match
- 0.15 rating score
- 0.15 deal score
- 0.10 distance score
- 0.10 friend signal
- 0.05 verification score

Each subscore should be from 0 to 100.

Result Card Explanation

Every recommendation should explain itself.

Example:

Match: 92% Recommended because you rank ramen highly, this place has a student deal, it is within 3 km, and two saved users rated it highly.

Acceptance Criteria

- Recommendations are ranked.
 - Match score is displayed.
 - Reasons are displayed.
 - User can save or claim from recommendation card.
 - User can filter recommendations.
-

7.6 Food Finder Flow

Purpose

Give users a simple flow for finding a place to eat.

Entry Point

Button:

Find Food

Guided Inputs

Sentence builder:

I want [food type] for [occasion] within [distance] for [budget].

Fields:

- Food type
- Occasion
- Distance
- Budget
- Time
- Preferences

Food type options:

- Any
- Burgers
- Sushi
- Ramen
- Pizza
- Cafe
- Dessert
- Bubble tea
- Vegetarian
- Breakfast
- Cheap eats

Occasion options:

- Quick lunch
- After school
- Study session
- Group dinner
- Date
- Dessert run
- Weekend plan
- Solo meal

Budget options:

- Under \$10
- Under \$15
- Under \$20
- \$20+
- No budget

Preferences:

- Student discount
- Open now
- Good for groups
- Quiet
- Vegetarian options
- Highly rated
- Active deal

Output

PalateMatch returns:

- Top 3 best matches

- Why each matched
 - Deal availability
 - Save and claim options
-

7.7 Group Decision Mode

Purpose

Help groups decide where to eat.

Flow

1. User creates group session.
2. Selects occasion, budget, distance, and food types.
3. Mock friends join from seeded profiles.
4. Each person votes yes/maybe/no on options.
5. App finds overlap.
6. App returns top group match.

Group Inputs

- Group size
- Occasion
- Budget
- Distance
- Dietary needs
- Food preferences

Voting UI

Cards with:

- Business name
- Cuisine
- Price
- Rating
- Deal
- Distance

Buttons:

- Yes
- Maybe
- No

Output

Group result:

Best group match: Seoul Bites 86% group fit Works for 4 of 5 users, under budget, vegetarian options available, active student deal.

Acceptance Criteria

- User can start group mode.
 - Mock friends can vote.
 - App calculates group fit.
 - App explains tradeoffs.
 - User can save or claim selected business/deal.
-

7.8 Deals and Coupons

Purpose

Help users support local food businesses through claimable deals.

Deal Types

- Student discount
- Limited-time deal
- Combo deal
- Group deal
- Happy hour
- New customer offer
- Loyalty offer
- Mystery box
- Free add-on
- Event special

Deal Card Fields

- Business name
- Deal title
- Price
- Original price
- Savings
- Valid time
- Claim limit
- Current claims
- Expiration
- Claim button

- Save button

Claim Flow

1. User clicks "Claim."
2. App checks eligibility.
3. App displays verification modal.
4. App generates claim code.
5. Claim appears in user claims page.
6. Business can redeem claim.

Claim Code Format

PAL-####

Example: PAL-4821

Claim Statuses

- Active
- Redeemed
- Expired
- Cancelled

Acceptance Criteria

- User can view deals.
- User can sort deals.
- User can claim deals.
- Claim code is generated.
- User cannot claim expired deal.
- User cannot claim same deal twice.
- Claim persists.

7.9 Verification and Bot Prevention

Purpose

Satisfy the bot-prevention requirement and prevent fake reviews/deal abuse.

Verification Methods

For MVP demo:

- Mock verification code
- Claim cooldown

- Duplicate claim prevention
- Review only after redemption
- Verified review badge
- Max active claims

Verification Modal

Title:

Quick verification

Fields:

- Checkbox: "I agree to only claim deals I intend to use."
- Code input: user types "2468"

Rules:

- Must check box.
- Must enter correct code.
- Max 3 attempts.
- Store verifiedHuman true for session.

Anti-Abuse Rules

- Max 3 active claims per user.
- Same user cannot claim same deal twice.
- Reviews require redeemed claim.
- Same claim cannot be reviewed twice.
- Optional review text has length limits.

7.10 Reviews and Ratings

Purpose

Allow users to rate businesses while keeping reviews trustworthy.

Review Unlock Rule

User can review after:

- claiming a deal and business marks it redeemed, or
- manually marking a seeded visit as completed in demo mode.

Review Fields

- Rating from 1 to 5
- Text review
- Tags
- Would recommend toggle
- Visit type

Tags:

- Good value
- Friendly staff
- Fast service
- Great taste
- Student-friendly
- Clean
- Good portion
- Good for groups
- Good for studying
- Vegetarian options

Validation

- Rating required.
- Rating must be between 1 and 5.
- Text minimum 10 characters.
- Text maximum 300 characters.
- User cannot review same claim twice.
- Review must attach to a business.

Business Rating Update

After a review:

- Recalculate average rating.
- Increase review count.
- Add review to business profile.

7.11 Saved Page

Purpose

Let users bookmark food businesses and deals.

Tabs

- Saved Places

- Saved Deals

Saved Places Sort

- Category
- Rating
- Recently saved
- Active deals
- Distance
- Personal rank

Saved Deals Sort

- Ending soon
- Highest savings
- Business rating
- Cuisine
- Recently saved

Acceptance Criteria

- User can save business.
 - User can unsave business.
 - User can save deal.
 - User can unsave deal.
 - Saved data persists.
-

7.12 Rankings Page

Purpose

Show the user's personal ranked food lists.

Lists

- Overall favorites
- Best burgers
- Best sushi
- Best ramen
- Best cafes
- Best dessert
- Best study spots
- Best cheap eats
- Best value

Ranking Card

Fields:

- Rank number
- Business name
- Cuisine
- Rating
- Personal notes
- Last ranked date
- Move/re-rank button
- View profile button

Actions

- Add place to ranking
 - Re-rank
 - Remove from ranking
 - Filter by category
-

7.13 Taste Profile Page

Purpose

Show a summary of the user's food preferences.

Metrics

- Top cuisines
- Highest ranked categories
- Average rating given
- Most visited category
- Most saved category
- Favorite price range
- Deal claims
- Ranking confidence

Example Output

Your taste leans toward ramen, cafes, and dessert spots. You tend to prefer student-friendly places under \$20 with strong value ratings.

Data Sources

- Rankings
- Reviews

- Saves
 - Deal claims
 - Food finder inputs
-

7.14 User Reports Page

Purpose

Show meaningful data analysis for the user.

Filters

- Date range
- Category
- Cuisine
- Claim status
- Rating range

Metrics

- Restaurants visited
- Deals claimed
- Deals redeemed
- Estimated money saved
- Businesses supported
- Reviews submitted
- Average rating given
- Top cuisine
- Top ranked business

Charts

- Claims by category
- Savings over time
- Ratings distribution
- Businesses supported per month
- Favorite cuisines

Acceptance Criteria

- Reports update from user actions.
 - Reports can be filtered.
 - Reports are clear and presentable.
 - Charts use local data.
-

8. Business Owner Features

8.1 Business Dashboard

Purpose

Give food businesses an overview of engagement.

Metrics

- Active deals
- Profile views
- Saves
- Claims
- Redemptions
- Average rating
- Review count
- Conversion rate
- Most popular deal

Charts

- Claims over time
 - Redemptions by deal
 - Rating trend
 - Review tag frequency
 - Saves by week
-

8.2 Create Deal Page

Purpose

Let businesses create structured deals.

Fields

- Deal title
- Deal type
- Description
- Price
- Original price
- Valid from
- Valid until
- Max claims
- Student-only toggle

- Tags
- Verification required toggle

Validation

- Title required.
 - Title max 80 characters.
 - Description max 250 characters.
 - Price must be non-negative.
 - Original price must be greater than deal price.
 - End time must be after start time.
 - Max claims must be at least 1.
 - Category must match business category.
-

8.3 Manage Deals Page

Tabs

- Active
- Scheduled
- Expired

Deal Card Fields

- Title
 - Price
 - Dates
 - Claims
 - Redemptions
 - Views
 - Status
 - Edit button
 - Deactivate button
-

8.4 Redeem Claim Page

Purpose

Let a business verify and redeem a user claim.

Input

- Claim code

Validation

- Code must match PAL-####
- Code must exist.
- Claim must belong to that business.
- Claim must be active.
- Claim must not be expired.
- Claim must not already be redeemed.

Success State

Claim redeemed. The user can now leave a verified review.

Error States

- Invalid code
 - Expired claim
 - Already redeemed
 - Wrong business
-

8.5 Business Analytics Page

Metrics

- Deal views
- Claims
- Redemptions
- Conversion rate
- Average rating
- Saves
- Review tags
- Repeat customers
- Estimated revenue influenced

Formula Examples

Conversion rate = redeemed claims divided by deal views.

Estimated revenue influenced = redeemed claims times deal price.

9. Data Model

Use TypeScript types.

User

```
export type UserRole = "customer" | "businessOwner" | "admin";

export type User = {
  id: string;
  name: string;
  email: string;
  role: UserRole;
  verified: boolean;
  homeLocationId: string;
  preferences: UserPreferences;
  createdAt: string;
};
```

UserPreferences

```
export type UserPreferences = {
  favoriteCuisines: Cuisine[];
  maxDefaultDistanceKm: number;
  defaultBudgetMax?: number;
  studentDiscountPreferred: boolean;
  dietaryPreferences: string[];
  savedBusinessIds: string[];
  savedDealIds: string[];
};
```

Cuisine

```
export type Cuisine =
  | "burgers"
  | "pizza"
  | "sushi"
  | "ramen"
  | "cafe"
  | "dessert"
  | "bubbleTea"
  | "breakfast"
  | "vegetarian"
  | "bakery"
  | "fastCasual"
  | "other";
```

FoodBusiness

```
export type FoodBusiness = {
  id: string;
  name: string;
  category: "restaurant" | "cafe" | "bakery" | "dessert" | "foodTruck" |
  "bubbleTea";
  cuisine: Cuisine;
  description: string;
  address: string;
  location: GeoPoint;
  hours: BusinessHours[];
  ratingAverage: number;
  reviewCount: number;
  verified: boolean;
  priceLevel: 1 | 2 | 3 | 4;
  tags: string[];
  accessibilityFeatures: string[];
  ownerUserId: string;
  createdAt: string;
};
```

GeoPoint

```
export type GeoPoint = {
  lat: number;
  lng: number;
};
```

BusinessHours

```
export type BusinessHours = {
  dayOfWeek: number;
  openTime: string;
  closeTime: string;
};
```

Deal

```
export type DealType =
  | "studentDiscount"
  | "limitedTime"
```

```

    | "combo"
    | "groupDeal"
    | "happyHour"
    | "newCustomer"
    | "loyalty"
    | "mysteryBox"
    | "freeAddon";

export type Deal = {
  id: string;
  businessId: string;
  title: string;
  description: string;
  dealType: DealType;
  price: number;
  originalPrice?: number;
  validFrom: string;
  validUntil: string;
  maxClaims: number;
  currentClaims: number;
  views: number;
  tags: string[];
  studentOnly: boolean;
  active: boolean;
  verificationRequired: boolean;
  createdAt: string;
};

```

FoodFinderRequest

```

export type Occasion =
  | "quickLunch"
  | "afterSchool"
  | "studySession"
  | "groupDinner"
  | "date"
  | "dessertRun"
  | "weekendPlan"
  | "soloMeal";

export type FoodFinderRequest = {
  id: string;
  userId: string;
  cuisine?: Cuisine;
  occasion: Occasion;
  budgetMax?: number;
};

```

```

    distanceKm: number;
    timeStart: string;
    timeEnd: string;
    preferences: string[];
    status: "draft" | "submitted" | "matched" | "expired";
    verifiedHuman: boolean;
    createdAt: string;
  };

```

RecommendationResult

```

export type RecommendationResult = {
  businessId: string;
  dealId?: string;
  requestId?: string;
  score: number;
  scoreBreakdown: PalateScoreBreakdown;
  reasons: string[];
};

```

PalateScoreBreakdown

```

export type PalateScoreBreakdown = {
  tasteScore: number;
  cuisineScore: number;
  ratingScore: number;
  dealScore: number;
  distanceScore: number;
  friendScore: number;
  verificationScore: number;
};

```

Claim

```

export type ClaimStatus = "active" | "redeemed" | "expired" | "cancelled";

export type Claim = {
  id: string;
  userId: string;
  dealId: string;
  businessId: string;
  claimCode: string;
  status: ClaimStatus;
};

```

```
    createdAt: string;
    expiresAt: string;
    redeemedAt?: string;
};
```

Review

```
export type Review = {
  id: string;
  userId: string;
  businessId: string;
  dealId?: string;
  claimId?: string;
  rating: number;
  text: string;
  tags: string[];
  verified: boolean;
  createdAt: string;
};
```

SavedBusiness

```
export type SavedBusiness = {
  userId: string;
  businessId: string;
  savedAt: string;
  tags: string[];
};
```

SavedDeal

```
export type SavedDeal = {
  userId: string;
  dealId: string;
  savedAt: string;
};
```

PersonalRanking

```
export type PersonalRanking = {
  userId: string;
```

```
rankingType: "overall" | "cuisine" | "occasion" | "value";
cuisine?: Cuisine;
occasion?: Occasion;
rankedBusinessIds: string[];
updatedAt: string;
};
```

10. Algorithms and Logic

10.1 PalateMatch Recommendation Algorithm

Purpose

Rank local food businesses based on personal taste and context.

Formula

Score =
0.25 taste score

- 0.20 cuisine score
- 0.15 rating score
- 0.15 deal score
- 0.10 distance score
- 0.10 friend score
- 0.05 verification score

Each score should be from 0 to 100.

Taste Score

Based on:

- user rankings
- user reviews
- saved places
- previous claims
- preferred cuisines

Cuisine Score

- 100 if exact cuisine match
- 70 if related cuisine
- 40 if broad food match
- 0 if irrelevant

Rating Score

Convert 1 to 5 star rating into 0 to 100.

Example: A 4.5 rating becomes 90.

Deal Score

- 100 if active deal fits user budget
- 80 if active deal exists
- 50 if no deal but strong rating
- 0 if deal expired

Distance Score

- 100 if very close
- Scales down as distance approaches selected max
- 0 if outside max distance

Friend Score

Use seeded friend data:

- 100 if highly ranked by close friend
- 70 if saved by friend
- 40 if reviewed by friend
- 0 if no friend signal

Verification Score

- 100 if verified business and verified reviews
- 70 if verified business
- 40 if unverified business

Recommendation Reasons

Generate reasons like:

- "Matches your top cuisine"
 - "Ranked highly by users with similar taste"
 - "Has an active student deal"
 - "Within your distance range"
 - "Highly rated by verified reviews"
 - "Saved by your friend Maya"
-

10.2 TasteRank Binary Insertion

Purpose

Efficiently place a business into a user's ranked list.

Process

1. Get user's ranked list for selected category.
2. Compare new business to middle business.
3. User chooses better, worse, about same, or skip.
4. Narrow search range.
5. Repeat until insertion position is found.
6. Insert business into ranked array.
7. Save ranking.

Complexity

Binary insertion uses about log base 2 of n comparisons.

Acceptance Criteria

- Works with empty rankings.
 - Works with single-item rankings.
 - Works with larger seeded lists.
 - Handles "about same."
 - Allows skip.
 - Persists updated order.
-

10.3 Review Verification

A review is verified if:

- User has redeemed claim, or
 - Demo visit is marked completed.
 - Review is connected to business.
 - No duplicate review exists for same claim.
-

10.4 Claim Code Generation

Format: PAL-####

Rules:

- Unique among active claims.
 - Tied to user, business, and deal.
 - Expires with the deal.
 - Redeemable once.
-

10.5 Report Calculations

Estimated savings: Sum of original price minus deal price for redeemed deals.

Business conversion rate: Redeemed claims divided by deal views.

Average rating: Sum of ratings divided by review count.

Taste confidence: Based on number of rankings, reviews, and saved businesses.

11. Input Validation

Food Finder Validation

Required:

- occasion
- distance
- time window

Optional but validated:

- cuisine
- budget
- preferences

Rules:

- Budget must be non-negative.
- Distance must be between 1 km and 25 km.
- End time must be after start time.
- Time cannot be in the past.
- Request cannot be duplicated within cooldown.
- Optional notes must be under 120 characters.
- Optional notes cannot contain links.

Deal Claim Validation

Rules:

- User must be verified.
- Deal must be active.
- Deal must not be expired.
- Max claims must not be reached.
- User cannot claim same deal twice.
- User cannot exceed 3 active claims.

Review Validation

Rules:

- Rating must be 1 to 5.
- Text must be 10 to 300 characters.
- Review must connect to a valid business.
- Verified review must connect to redeemed claim.
- Duplicate claim reviews are blocked.

Business Deal Validation

Rules:

- Title required.
- Price must be non-negative.
- Original price must exceed deal price.
- End time must be after start time.
- Max claims must be at least 1.
- Description max 250 characters.

12. Technical Requirements

Recommended Stack

- React
- TypeScript
- Vite
- LocalStorage or IndexedDB
- Recharts or Chart.js
- CSS Modules or Tailwind
- No required backend
- No required internet

Standalone Requirement

The app must run without live APIs. All demo data should be seeded locally.

Data Persistence

Persist:

- users
- businesses
- deals
- claims
- reviews
- saved businesses
- saved deals
- rankings
- food finder requests
- reports

Demo Reset

Include a reset button to restore original seeded data.

13. Recommended File Structure

```
src/  
  app/  
    App.tsx  
    routes.tsx  
    providers.tsx  
  
  models/  
    User.ts  
    FoodBusiness.ts  
    Deal.ts  
    FoodFinderRequest.ts  
    Claim.ts  
    Review.ts  
    Ranking.ts  
    Report.ts  
  
  data/  
    seedUsers.ts  
    seedBusinesses.ts
```

```
seedDeals.ts
seedClaims.ts
seedReviews.ts
seedRankings.ts

services/
  storageService.ts
  recommendationService.ts
  rankingService.ts
  claimService.ts
  reviewService.ts
  reportService.ts
  validationService.ts
  businessService.ts
  userService.ts

utils/
  distance.ts
  dateTime.ts
  formatting.ts
  sorting.ts
  ids.ts

components/
  layout/
    Sidebar.tsx
    TopBar.tsx
    MobileNav.tsx
    PageHeader.tsx

  common/
    Button.tsx
    Card.tsx
    Badge.tsx
    Modal.tsx
    EmptyState.tsx
    FormError.tsx
    HelpTooltip.tsx

  foodFinder/
    CuisineSelector.tsx
    OccasionSelector.tsx
    BudgetSelector.tsx
    DistanceSelector.tsx
    TimeWindowSelector.tsx
    PreferenceChips.tsx
    FoodFinderPreview.tsx
```

```
businesses/  
  BusinessCard.tsx  
  BusinessProfileHeader.tsx  
  BusinessReviews.tsx  
  ActiveDeals.tsx  
  
deals/  
  DealCard.tsx  
  ClaimCodeCard.tsx  
  DealFilters.tsx  
  
rankings/  
  ComparisonModal.tsx  
  RankingList.tsx  
  RankingCard.tsx  
  
reports/  
  MetricCard.tsx  
  SavingsChart.tsx  
  CuisineChart.tsx  
  RatingDistributionChart.tsx  
  ReportFilters.tsx  
  
pages/  
  HomePage.tsx  
  ExplorePage.tsx  
  FoodFinderPage.tsx  
  RecommendationsPage.tsx  
  BusinessProfilePage.tsx  
  DealsPage.tsx  
  SavedPage.tsx  
  ClaimsPage.tsx  
  RankingsPage.tsx  
  TasteProfilePage.tsx  
  UserReportsPage.tsx  
  BusinessDashboardPage.tsx  
  CreateDealPage.tsx  
  ManageDealsPage.tsx  
  RedeemClaimPage.tsx  
  BusinessAnalyticsPage.tsx  
  HelpPage.tsx  
  DemoControlsPage.tsx  
  
styles/  
  globals.css  
  tokens.css
```

14. Service Function Requirements

recommendationService.ts

Functions:

- getRecommendations(userId, request)
- calculatePalateScore(user, business, deal, request)
- calculateTasteScore(user, business)
- calculateCuisineScore(request, business)
- calculateRatingScore(business)
- calculateDealScore(deal, request)
- calculateDistanceScore(request, business)
- calculateFriendScore(user, business)
- calculateVerificationScore(business)
- generateRecommendationReasons(scoreBreakdown, user, business, deal, request)

rankingService.ts

Functions:

- getRanking(userId, rankingType, cuisine, occasion)
- startBinaryInsertion(userId, businessId, ranking)
- processComparison(answer)
- insertBusinessAtIndex(ranking, businessId, index)
- removeBusinessFromRanking(ranking, businessId)
- moveBusinessInRanking(ranking, businessId, newIndex)

claimService.ts

Functions:

- createClaim(userId, dealId)
- generateClaimCode()
- validateClaimCode(code)
- redeemClaim(code, businessId)
- expireOldClaims()
- getUserClaims(userId)
- getBusinessClaims(businessId)

reviewService.ts

Functions:

- canUserReview(userId, businessId, claimId)

- createReview(reviewInput)
- validateReview(reviewInput)
- getBusinessReviews(businessId)
- updateBusinessRating(businessId)

reportService.ts

Functions:

- getUserReport(userId, filters)
- getBusinessReport(businessId, filters)
- calculateEstimatedSavings(claims, deals)
- calculateConversionRate(views, redemptions)
- groupClaimsByCuisine(claims, businesses)
- groupSavingsByMonth(claims, deals)
- calculateTasteProfile(userId)

validationService.ts

Functions:

- validateFoodFinderRequest(request)
- validateDeal(deal)
- validateClaim(userId, dealId)
- validateReview(reviewInput)
- validateTimeWindow(start, end)
- validateOptionalNote(note)

storageService.ts

Functions:

- loadData()
 - saveData(data)
 - resetDemoData()
 - getCollection(collectionName)
 - updateCollection(collectionName, newData)
-

15. Seed Data Requirements

Users

Create at least:

- 3 customer users
- 2 business owner users
- 1 admin/demo user

Example users:

- Lucas Chen
- Maya Patel
- Ethan Wong
- Ava Singh
- Sam Rivera, business owner
- Nina Brooks, business owner

Food Businesses

Create at least 20 food businesses.

Minimum:

- Restaurants: 6
- Cafes: 4
- Bakeries: 3
- Dessert shops: 3
- Bubble tea: 2
- Food trucks: 2

Each business must have:

- name
- category
- cuisine
- description
- location
- hours
- rating
- review count
- tags
- verified status
- price level
- ownerUserId

Deals

Create at least 25 deals.

Examples:

- \$11.99 student lunch bowl
- \$3 iced latte after school
- Buy one get one bubble tea
- \$7 mystery pastry box
- 15 percent off ramen combo
- Group dessert deal
- Free topping with student ID
- Pizza slice combo under \$10
- Cafe study combo
- Weekend brunch discount

Reviews

Create at least 40 reviews.

Include:

- verified reviews
- unverified reviews
- ratings from 3 to 5
- realistic tags
- multiple businesses

Claims

Create:

- 5 active claims
- 5 redeemed claims
- 3 expired claims

Rankings

Create rankings for:

- overall favorites
- burgers
- cafes
- ramen
- dessert

- study spots
 - cheap eats
-

16. UX Design System

Visual Style

Clean, modern, food-focused, polished.

Mood

The app should feel:

- warm
- social
- personal
- easy
- polished
- trustworthy

Color Palette

Suggested:

- Primary: warm red, orange, or deep purple
- Background: off-white
- Cards: white
- Text: dark gray
- Success: green
- Warning: orange
- Error: red
- Deal badge: yellow or light orange
- Verified badge: blue

Typography

Use one clean sans-serif font.

Hierarchy:

- Page titles large and bold
- Section headings medium and bold
- Body text readable
- Helper text smaller and muted

Components

Use consistent:

- cards
- badges
- buttons
- chips
- modals
- tabs
- filters
- empty states
- chart cards

Accessibility

Include:

- keyboard navigable controls
 - visible focus states
 - alt text for images/icons
 - labels for form fields
 - clear error messages
 - sufficient color contrast
 - no icon-only buttons
 - reduced motion option if animations exist
-

17. Error and Empty States

No Recommendations

No strong matches found. Try widening your distance, changing your budget, or selecting another cuisine.

No Saved Places

You have not saved any places yet. Explore local restaurants or find food based on your taste.

No Deals

No active deals match this filter right now.

Invalid Claim

This claim code does not exist or is no longer active.

Expired Deal

This deal has expired. View similar active deals.

Review Locked

Verified reviews unlock after you redeem a deal or complete a visit.

Ranking Empty

Start ranking places by comparing restaurants you have tried.

18. Demo Script

7-Minute Demo Flow

1. Introduce Palate.
2. Explain the problem: generic food ratings do not capture personal taste.
3. Show home dashboard.
4. Use Food Finder: "cafe study spot under \$15 within 3 km."
5. Show PalateMatch recommendations.
6. Explain score and match reasons.
7. Save a business.
8. Claim a deal.
9. Show verification and claim code.
10. Switch to business dashboard.
11. Redeem claim code.
12. Switch back to user.
13. Leave verified review.
14. Use TasteRank comparison.
15. Show updated ranking.
16. Show Taste Profile.
17. Show user report.
18. Show business analytics.
19. Explain code architecture and algorithms.

Must Mention

- Standalone local data

- Categories and sorting
 - Reviews and ratings
 - Bookmarks
 - Deals and coupons
 - Verification
 - PalateMatch recommendation algorithm
 - TasteRank binary insertion
 - Reports and analytics
 - Modular code and TypeScript data models
-

19. Acceptance Criteria by Rubric Category

Code Quality

- TypeScript models exist for all major data objects.
- Code is modular.
- Services handle business logic.
- Components handle UI.
- Comments explain non-obvious algorithms.
- Naming conventions are clear.
- No runtime errors.

UX

- User journey is easy to follow.
- Navigation is clear.
- Instructions are built into the UI.
- Form errors are helpful.
- Help panel exists.
- Accessibility basics are included.
- UI has no spelling errors.

Functionality

- App addresses local business discovery.
- Users can sort by category.
- Users can leave reviews and ratings.
- Users can sort by ratings and reviews.
- Users can save businesses.
- Users can view and claim deals.
- Verification prevents bot-like behavior.
- Intelligent recommendations exist.
- Reports exist.

Data Storage

- Uses arrays/lists and typed objects.
- Stores businesses, deals, users, claims, reviews, saves, and rankings.
- Data updates after actions.
- Data persists after refresh.
- Variable scope is logical.

Reports

- User reports are customizable.
 - Business analytics are customizable.
 - Charts and metrics update from local data.
 - Reports show meaningful analysis.
-

20. Build Order for AI Agent

Phase 1: Setup

1. Create React + TypeScript + Vite app.
2. Add routing.
3. Add layout.
4. Add design tokens.
5. Add seed data.
6. Add local storage service.

Phase 2: Data Models and Services

1. Define models.
2. Build storage service.
3. Build recommendation service.
4. Build ranking service.
5. Build claim service.
6. Build review service.
7. Build report service.
8. Build validation service.

Phase 3: User App

1. Home page.
2. Explore page.
3. Food Finder page.
4. Recommendations page.
5. Business profile page.

6. Deals page.
7. Saved page.
8. Claims page.

Phase 4: Rankings and Taste

1. TasteRank comparison modal.
2. Rankings page.
3. Taste profile page.
4. Recommendation explanations.

Phase 5: Business App

1. Business dashboard.
2. Create deal page.
3. Manage deals page.
4. Redeem claim page.
5. Business analytics page.

Phase 6: Reports and Polish

1. User reports page.
2. Business charts.
3. Empty states.
4. Error states.
5. Help panel.
6. Accessibility check.
7. Reset demo data.
8. Final QA.

21. Final MVP Checklist

Before submission, Palate must support:

- Browse local food businesses
- Filter by category/cuisine
- Sort by rating
- Sort by review count
- Save businesses
- Save deals
- View business profiles
- View active deals
- Claim deal
- Generate claim code
- Redeem claim as business

- Leave verified review
 - Update average rating
 - Use Food Finder
 - Get PalateMatch recommendations
 - See match score and reasons
 - Use TasteRank comparisons
 - Update personal rankings
 - View taste profile
 - View user reports
 - View business analytics
 - Persist local data
 - Reset demo data
 - Run offline
 - Avoid runtime errors
-

22. Final Product Summary

Palate is a personalized local food discovery app that helps users discover, rank, compare, save, review, and support restaurants, cafes, bakeries, dessert shops, and other food businesses. Its core intelligent features are PalateMatch, which recommends places based on taste and context, and TasteRank, which builds personal rankings through pairwise comparisons. The app also includes deals, coupon claiming, verification, verified reviews, user reports, and business analytics.

Palate should win on polish, personalization, ranking logic, and clear local business support.